

Assignment 04

For this assignment you will be writing a series of programs which should be saved independently as their own ".py" file. **Remember to include comments and name files appropriately.**

Problem 0: Seeing Stars

Part 1

Write a program that prompts the user for two numbers. You can always assume the user will supply integers when prompted, but you cannot assume the range of these integers. Ensure that both integers are positive, and that the second integer is larger than the first. For example:

```
Number 1: -5
Invalid, try again
Number 1: 5
Number 2: -5
Invalid, try again
Number 2: 4
Invalid, try again
Number 2: 6
```

Part 2

Next, generate a triangle pattern that looks like the following. Note that the first line you should print should contain the first number the user supplied, followed by that many stars. The next line should increase by 1 and repeat the process until you reach the second number.

```
Number 1: 1
Number 2: 9

1 *
2 **
3 ***
4 ****
5 *****
6 *
7 *
8 *
9 *
```

Part 3

Generate the bottom half of the triangle so that it looks like an "arrow". For this part of the shape the numbers should decrease and return back to the first number. Here are a few runnings of your program - user input is underlined:

```
Number 1: 1
Number 2: 9
1 *
2 **
3 ***
4 ****
5 *****
6 *
7 *
8 *
9 *
8 *
7 *
6 *
5 *
4 *
3 *
2 *
1 *
Number 1: -10
Invalid, try again
Number 1: -20
Invalid, try again
Number 1: 5
Number 2: -5
Invalid, try again
Number 2: 3
Invalid, try again
Number 2: 4
Invalid, try again
```

Number 2: 5

Invalid, try again

Number 2: 20

5 *****

6 *****

7 *****

8 *****

9 *****

10 *****

11 *****

12 *****

13 *****

14 *****

15 *****

16 *****

17 *****

18 *****

19 *****

20 *****

19 *****

18 *****

17 *****

16 *****

15 *****

14 *****

13 *****

12 *****

11 *****

10 *****

9 *****

8 *****

7 *****

6 *****

Problem 0 Points (15):

- Only accepts a positive number for first user input (3 points)
- Only accepts a positive number for second user input and second number must be larger than first number (6 points)
- Prints number and correct number of stars on each line, starting with number 1, going up to number 2, and going back down to number 1 (6 points)

Problem 1: Roll the Dice

In games with two dice (such as Monopoly), rolling **two 6-sided dice** and getting the **same side or number** on both dice is called "**doubles**" (i.e. the first die rolls a 5 and the second die rolls a 5 at the same time). One special type of "doubles" roll is called "snake eyes," which is rolling two 1's at the same time.

The most common size of dice used in games today is the 6-sided dice, but there are plenty of other sizes available.

For this assignment you will be writing a program which first prompts the user for the number of **sides to the dice they will be rolling**. Your user should be limited to selecting dice with the following number of sides:

- 4 sides
- 6 sides
- 8 sides

You can assume that that your user will **enter integers when prompted**. You will need to **validate** their input before you continue (i.e. entering -10 should cause your program to tell the user that their input is invalid). You should re-prompt the user to **enter a value if they supply bad data**.

Next, your program should keep "rolling the dice" until the program generates a "snake eyes" roll (1 and 1). This means that you will **roll two virtual dice of the specified size at a time**. The program should **"announce" every pair rolled** and tell the user if the roll qualifies as one of the following:

- A double roll (i.e. (2 and 2), (4 and 4), (6 and 6))
- A high roll (i.e. on a six sided die, (6 and 6))
- A high/low roll (i.e. on a six sided die, (1 and 6) or (6 and 1)
- An even roll (i.e. both die values are even: (2 and 2), (4 and 4))
- An odd roll (i.e. both die values are odd: (3 and 3), (5 and 5))
- A sum value (i.e. adding both die values together equals the size of the current die
- Snake Eyes (1 and 1) - this should end the rolling portion of the program

At the end of the game you will want to calculate the **average roll for each die** and present this information to the user. See the sample program below to see what this should look like. You should **format this number to two decimal places**. Here's a sample running of the program (user input is underlined for your reference - you do not need to code your program to generate underlines):

How many sides on your dice (4, 6, 8)? 13

Invalid size, try again.

How many sides on your dice (4, 6, 8)? 2

Invalid size, try again.

How many sides on your dice (4, 6, 8)? -1

Invalid size, try again.

How many sides on your dice (4, 6, 8)? 6

Thanks, here we go!

die #1 is *3* and die #2 is *2*

die #1 is *1* and die #2 is *5* Odds! Sum value is size value!

die #1 is *3* and die #2 is *5* Odds!

die #1 is *3* and die #2 is *3* Odds! Doubles! Sum value is size value!

die #1 is *1* and die #2 is *2*

die #1 is *4* and die #2 is *2* Evens! Sum value is size value!

die #1 is *5* and die #2 is *1* Odds! Sum value is size value!

die #1 is *5* and die #2 is *5* Odds! Doubles!

die #1 is *4* and die #2 is *3*

die #1 is *3* and die #2 is *2*

die #1 is *6* and die #2 is *5*

die #1 is *4* and die #2 is *1*

die #1 is *4* and die #2 is *1*

die #1 is *2* and die #2 is *5*

die #1 is *2* and die #2 is *4* Evens! Sum value is size value!

die #1 is *6* and die #2 is *1* High / Low!

die #1 is *6* and die #2 is *5*

die #1 is *1* and die #2 is *2*

die #1 is *6* and die #2 is *3*

die #1 is *3* and die #2 is *6*

die #1 is *1* and die #2 is *2*

die #1 is *1* and die #2 is *1* Odds! Doubles! Snake Eyes!

You finally got snake eyes on roll # 22

Along the way you rolled DOUBLES 3 times. (13.64% of all rolls were doubles)

Along the way you rolled TWO HIGH VALUES 0 times (0.00% of all rolls were two high values)

Along the way you rolled TWO EVENS 2 times. (9.09% of all rolls were two evens)

Along the way you rolled TWO ODDS 6 times. (27.27% of all rolls were two odds)

Along the way you rolled HIGH / LOW 1 times. (4.55% of all rolls were high/low)

Along the way you rolled A SUM VALUE 5 times. (22.73% of all rolls were a sum value)

Average roll for die #1: 3.36

Average roll for die #2: 3.00

Note that the "snake eyes" roll that causes your loop to end should also count as a "doubles" roll.

Problem 1 Points (30):

- Only accepts 4, 6, or 8 as input for the number of sides (3 points)
- Game continues until snake eyes (1 and 1) are rolled (4 points)
- You may have to run the program a few times to test this, but:
 - If two of the same number are rolled, "doubles" is printed (2 points)
 - If two of the highest possible number are rolled, "high roll" is printed (2 points)
 - If the highest number is rolled along with a 1, "high/low roll" is printed (2 points)
 - If both dice are odd numbers, "odd" is printed (2 points)
 - If both dice are even numbers, "even" is printed (2 points)
 - If snake eyes (two 1's) are rolled, "snake eyes" is printed and the game ends (2 points)
 - NOTE: if one roll is more than one of the above, both should be printed (e.x. 6 and 6 would print "high roll" and "evens")
- The number of "special rolls" above are printed at the bottom along with their percentages (3 points)
- The average roll for each dice is printed at the end and formatted to 2 decimal points (3 points)
- Format of output is generally followed (special rolls printed on same line of each roll) (5 points)

Problem 2: 21

For this problem you will be creating a simplified version of the card game 21 with a player and a dealer. Here's how it will work:

- The aim of the game is to get 21 or as close to 21 as possible
- To simplify the game, we will have only the number cards and the ace as possible options. Ace is worth 11 points and can be printed as 11.
- The dealer and player each start with 2 cards. The dealer has one card face up and one card face down.
- The player sees his cards and decides if he wants another card (hit) or will stay with his hand.
 - If the player has or gets to exactly 21, the game ends and the player wins without the dealer's cards being revealed.
 - If the player has or goes over 21, the game ends and the player loses without the dealer's cards being revealed.
- The dealer then turns over the other card.
 - If the dealer has 16 or under they must take another card. The dealer may only do this one time.
 - If the dealer has 21 (Ace and 10) the dealer wins.
 - If the dealer goes over 21 then the player wins.
- Whoever is closest to 21 after the dealer's turn wins. If they have the same score, they tie.

Here are some sample runnings of the game with user input underlined:

Let's Play 21...

Here are the dealer's cards: 4 X

Turn: Player

Here are your cards: 10 8

What would you like to do? (Hit/Stay) Stay

Turn: Dealer

Here are your cards: 4 10

Your cards total to 16 or less. You must take another card.

Here are your cards: 4 10 6

The Player has 18 and the Dealer has 20.

The Dealer wins!

Let's Play 21...

Here are the dealer's cards: 8 X

Turn: Player

Here are your cards: 11 2

What would you like to do? (Hit/Stay) Hit

Here are your cards: 11 2 8

The Player has 21!

The Player wins!

Let's Play 21...

Here are the dealer's cards: 6 X

Turn: Player

Here are your cards: 9 10

What would you like to do? (Hit/Stay) Stay

Turn: Dealer

Here are your cards: 6 9

Your cards total to 16 or less. You must take another card.

Here are your cards: 6 9 10

The Dealer goes bust!

The Player wins!

Here are some hints to get you started:

- You will be using randint to choose cards for the player and the dealer
- You may want to use a variable to keep track of the winner (note: it will start as 'None' which you can use as a placeholder)
- You may want to use break to end a turn early
- You can assume that the player will only enter hit or stay. However, the program should work regardless of capitalization.
- You will need to keep track of the card sum for both the dealer and the player as you go
- As long as the player has less than 21, he should have the option to continue drawing cards
- In order to account for the fact that the player can draw multiple cards, which you will need to print out, you will need to store this information somewhere. You do not know how many cards you will need in advance (so you cannot store the value of each potential new card in a variable). One way to do this would be to save the string you are printing as a variable and use string concatenation when the player draws a new card.

Problem 2 Points (30):

- Cards are randomly selected for both the player and the dealer before the game starts (2 points)
- There are variables that keep track of the sum of the player's cards and the dealer's cards (4 points)

- The dealer's cards are printed before the game starts; one is the card value, and one is an "X" as a placeholder (2 points)
- The player is allowed to keep drawing cards as long as the sum of his cards is < 21 (3 points)
- The input for the player's move is accepted regardless of capitalization (2 points)
- If the player decides to stay, his turn ends (3 points)
- The player's cards are correctly printed regardless of how many cards he has (3 points)
- If the player gets a score of ≥ 21 , the dealer does not get a turn (2 points)
- If player sum is equal to 21, "The player has 21!" is printed and the player wins (2 point)
- If player sum is > 21 , "The player goes bust!" is printed and the dealer wins (2 point)
- If dealer sum is ≤ 16 , the dealer draws another card which is added to his sum and printed out (3 points)
- If neither player or dealer gets 21, both scores are printed out at the end. Whoever has the higher score wins, otherwise "it's a tie!" is printed (2 points)

Problem 3: Math Quiz

For this program you will be writing some code that will ask the user 5 simple math problems (using addition, subtraction, or multiplication). The numbers chosen for the problems should be integers between 0 and 10 and be randomly chosen for each question. The math operation should also be randomly chosen for each question. You can assume that the user will supply a number.

At the end of each question, let the user know if they were correct or incorrect. Keep track of how many questions the user gets right. At the end, present them with a score and a letter grade based on the following conversions:

- 5/5 – A
- 4/5 – B
- 3/5 – C
- 2/5 – D
- $< 2/5$ – F

Here is a sample running of the program. Underlined numbers are user input:

```
Question 1. 5 + 6
What is the answer? 11
Correct!
Question 2. 7 - 9
What is the answer? 2
Incorrect!
Question 3. 10 * 4
What is the answer? 40
Correct!
```

Question 4. $2 * 0$

What is the answer? 0

Correct!

Question 5. $10 + 10$

What is the answer? 20

Correct!

You received 4 out of a possible 5 points.

You scored a B.

Use **variables** whenever possible. For example, in the above example, you would not directly print "You scored a B" but rather use **a variable for the grade**. It also makes sense to use variables for lines of text that get repeated.

Problem 3 Points (25):

- A **loop is used to keep track of question number** and control when the program ends (5 points)
- Numbers and operators **(+, -, *) are** randomly chosen for each of the 5 questions (5 points)
- If the question is answered correctly, "Correct!" is printed otherwise "Incorrect!" is printed (2 points)
- If the question is answered correctly, **a point is added to the user's score** (5 points)
- The correct score out of 5 is printed at the end (5 points)
- The score of 5 is correctly **converted to a letter grade at the end** (3 points)